Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій
Кафедра радіофізики та комп'ютерних технологій

Звіт

З лабораторної №3

З дисципліни «Професійний Python»

На тему **«Ітератори, генератори та потокова обробка даних»**

Виконав:
Студент 3 курсу
групи ФеП-31
Войтович М.А.
Перевірив:
проф. Свелеба С. А.

Львів 2026

Варіант: 4 (Потокова обробка системи банківських рахунків)

Мета: Сформувати практичні навички роботи з ітераційною моделлю Python, ітераторами, генераторами, ключовим словом yield, генераторними виразами (generator expressions), модулем itertools, принципом ледачих обчислень (lazy evaluation) та потоковою обробкою великих наборів даних без повного завантаження у пам'ять.

Теоретичні відомості

Iterable та Iterator:

Iterable — об'єкт, з якого за допомогою функції iter() можна отримати ітератор (наприклад, списки, файли тощо).

Iterator — об'єкт, який зберігає поточний стан обходу, реалізує метод __next__() і при завершенні елементів піднімає виняток StopIteration.

Власний ітератор та протокол ітерації:

Клас ітератора повинен реалізовувати два методи: __iter__(self), який повертає сам ітератор, та __next__(self), який повертає наступне значення або викликає StopIteration.

Генератори та ключове слово yield:

Функція-генератор містить yield замість return. Виклик такої функції повертає об'єкт-генератор, не виконуючи весь код одразу.

yield призупиняє виконання функції, зберігає локальні змінні та поточну позицію, повертаючи чергове значення.

Делегування генерації (yield from):

Дозволяє просто і ефективно делегувати генерацію значень іншому ітерабельтному об'єкту або рекурсивному генератору.

Генераторні вирази (Generator Expressions):

Записуються у круглих дужках (x for x in ...) і обчислюють значення «на вимогу», забезпечуючи мінімальне споживання оперативної пам'яті (lazy evaluation) на відміну від list comprehensions (eager evaluation).

Модуль itertools:

Набір оптимізованих будівельних блоків для ітераторів, що містить такі функції як islice (зрізи без матеріалізації), groupby (групування), accumulate (накопичувальні суми), pairwise (пари сусідніх елементів) та chain (об'єднання потоків).

Потокова обробка (Streaming) та Batch Processing:

Читання та обробка великих файлів (наприклад, понад 100 000 рядків) частинами (рядок за рядком або пакетами/batches), що запобігає переповненню оперативної пам'яті.

Постановка задачі

Розширити проєкт **Система банківських рахунків (Варіант 4)** шляхом впровадження ітераційної моделі та потокової обробки даних.

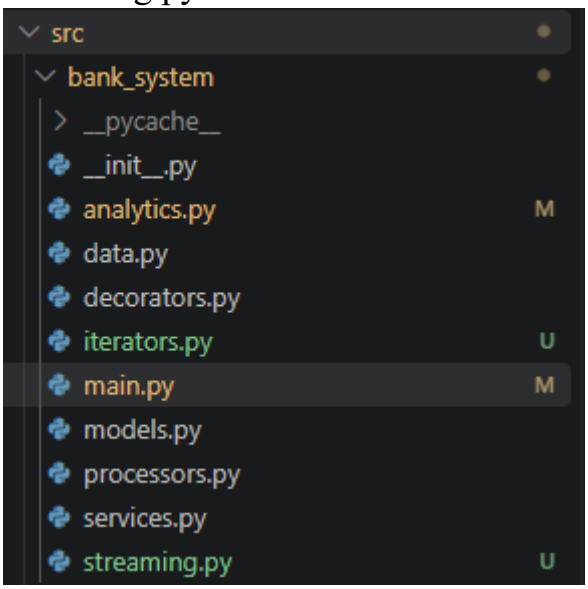
Основні вимоги:

1. Реалізувати власний ітератор (AccountRangeIterator) для генерації послідовностей.
2. Створити рекурсивний генератор із використанням yield from.
3. Реалізувати потоковий конвеєр (*lazy pipeline*): читання файлу (stream_csv_reader), парсинг і валідація, фільтрація за клієнтом/балансом, обробка операцій (поповнення/списання з контролем від'ємного балансу).
4. Задіяти інструменти модуля itertools (islice, groupby, accumulate, pairwise, chain).
5. Реалізувати пакетну обробку даних (*batch processing*).
6. Обробити набір даних обсягом не менше 100 000 записів.
7. Провести порівняння продуктивності та пікового споживання пам'яті між *Eager* (Pandas) та *Lazy* (Generators) підходами.

Хід виконання

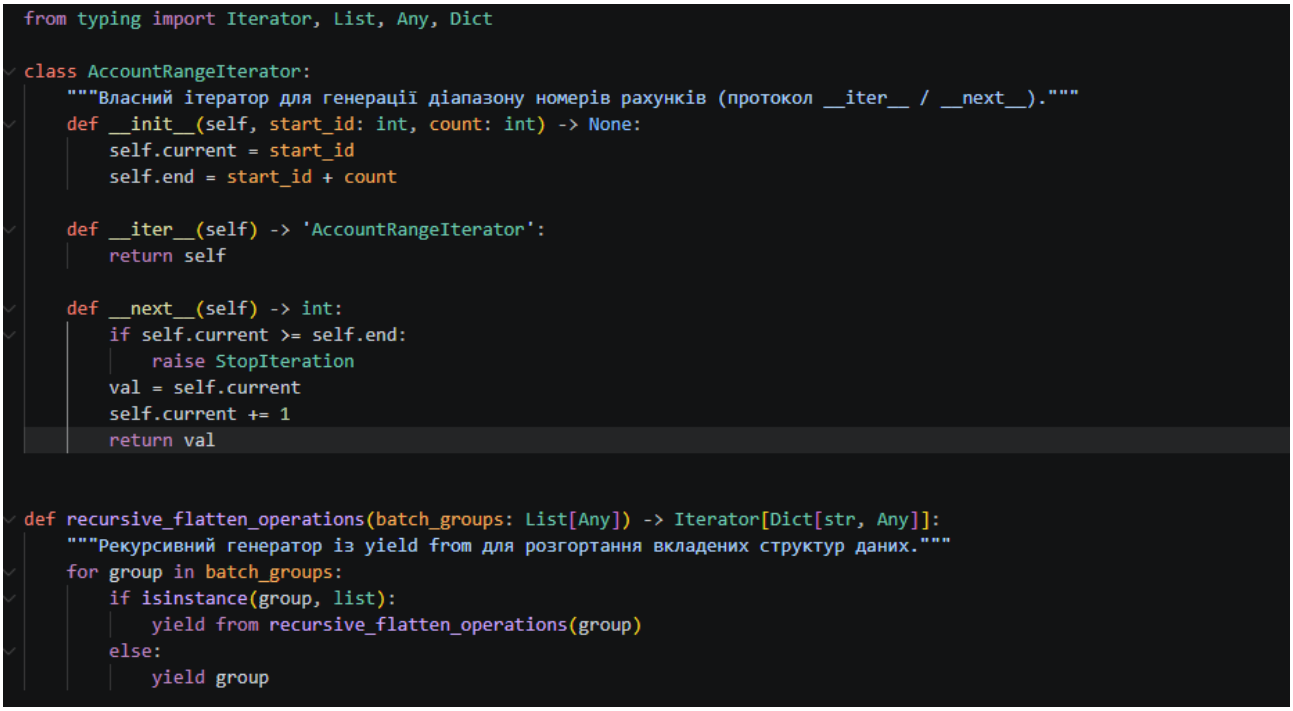
1. Структура проєкту (src-layout)

Проєкт організовано як Python-пакет, до якого додано нові модулі iterators.py та streaming.py:



2. Реалізація ключових модулів

src/bank_system/iterators.py: Містить власний ітератор AccountRangeIterator із протоколом __iter__ / __next__ та рекурсивний генератор recursive_flatten_operations з використанням yield from.



```python
from typing import Iterator, List, Any, Dict

class AccountRangeIterator:
    """Власний ітератор для генерації діапазону номерів рахунків (протокол __iter__ / __next__)."""
    def __init__(self, start_id: int, count: int) -> None:
        self.current = start_id
        self.end = start_id + count

    def __iter__(self) -> 'AccountRangeIterator':
        return self

    def __next__(self) -> int:
        if self.current >= self.end:
            raise StopIteration
        val = self.current
        self.current += 1
        return val


def recursive_flatten_operations(batch_groups: List[Any]) -> Iterator[Dict[str, Any]]:
    """Рекурсивний генератор із yield from для розгортання вкладених структур даних."""
    for group in batch_groups:
        if isinstance(group, list):
            yield from recursive_flatten_operations(group)
        else:
            yield group
```

src/bank_system/streaming.py: Реалізує генератор тестового датасету (100 000+ записів), потоковий зчитувач (stream_csv_reader), валідацію, фільтрацію та бізнес-логіку обробки транзакцій без завантаження всього файлу в оперативну пам'ять.

```python
import csv
import os
from typing import Iterator, Dict, Any
from bank_system.iterators import AccountRangeIterator

def generate_large_dataset(filepath: str, total_records: int = 100_000) -> None:
    """Генерує тестовий CSV-файл із банківськими операціями (> 100 000 записів)."""
    os.makedirs(os.path.dirname(filepath), exist_ok=True)
    clients = ["Client_A", "Client_B", "Client_C", "Client_D", "Client_E"]
    operations = ["deposit", "withdraw"]

    # Використання власного ітератора
    id_iter = AccountRangeIterator(start_id=1000, count=500)
    account_pool = list(id_iter)

    with open(filepath, mode="w", encoding="utf-8", newline="") as f:
        writer = csv.writer(f)
        writer.writerow(["account_number", "client", "balance", "operation", "amount"])

        for i in range(total_records):
            acc = f"ACC-{account_pool[i % len(account_pool)]}"
            client = clients[i % len(clients)]
            balance = round(1000.0 + (i * 7) % 50000.0, 2)
            operation = operations[i % 2]
            amount = round(10.0 + (i * 3) % 500.0, 2)
            writer.writerow([acc, client, balance, operation, amount])


def stream_csv_reader(filepath: str) -> Iterator[Dict[str, str]]:
    """Streaming reader: читає CSV-файл рядок за рядком за допомогою yield."""
    with open(filepath, mode="r", encoding="utf-8") as f:
        reader = csv.DictReader(f)
        for row in reader:
            yield row
```

```python
def parse_and_validate_records(rows: Iterator[Dict[str, str]]) -> Iterator[Dict[str, Any]]:
    """Парсить дані, перевіряє валідність та фільтрує пошкоджені записи."""
    for row in rows:
        try:
            account_number = str(row["account_number"]).strip()
            client = str(row["client"]).strip()
            balance = float(row["balance"])
            operation = str(row["operation"]).strip().lower()
            amount = float(row["amount"])

            if not account_number or not client or balance < 0 or amount < 0:
                continue
            if operation not in ["deposit", "withdraw"]:
                continue

            yield {
                "account_number": account_number,
                "client": client,
                "balance": balance,
                "operation": operation,
                "amount": amount
            }
        except (ValueError, KeyError):
            continue


def filter_accounts(records: Iterator[Dict[str, Any]], target_client: str = None, min_balance: float = 0.0) -> Iterator[Dict[str, Any]]:
    """Фільтрація рахунків за клієнтом або порогом балансу."""
    for record in records:
        if target_client and record["client"] != target_client:
            continue
        if record["balance"] < min_balance:
            continue
        yield record
```

```python
def process_account_operations(records: Iterator[Dict[str, Any]]) -> Iterator[Dict[str, Any]]:
    """Потокове застосування операцій з перевіркою на недопущення від'ємного балансу."""
    for record in records:
        bal = record["balance"]
        amt = record["amount"]
        if record["operation"] == "deposit":
            bal += amt
        elif record["operation"] == "withdraw":
            if bal - amt >= 0:
                bal -= amt
            else:
                continue

        updated_record = record.copy()
        updated_record["balance"] = round(bal, 2)
        yield updated_record


def run_lazy_pipeline(filepath: str, target_client: str = None, min_balance: float = 0.0) -> Iterator[Dict[str, Any]]:
    """Збирання повного lazy pipeline потокової обробки."""
    raw_lines = stream_csv_reader(filepath)
    parsed = parse_and_validate_records(raw_lines)
    filtered = filter_accounts(parsed, target_client=target_client, min_balance=min_balance)
    processed = process_account_operations(filtered)
    return processed
```

src/bank_system/main.py: Об'єднує конвеєр обробки, демонструє вибірку через itertools.islice, групування groupby, накопичувальні суми accumulate, а також здійснює порівняння часу виконання та пікового споживання пам'яті (*tracemalloc*) між *Eager* і *Lazy* підходами.

```python
import time
import tracemalloc
import os
from itertools import islice, groupby, accumulate, pairwise, chain

from bank_system.streaming import generate_large_dataset, run_lazy_pipeline

DATA_PATH = os.path.join("data", "input.csv")

def measure_eager_processing(filepath: str):
    """Eager evaluation: завантаження всього файлу в пам'ять (наприклад, через pandas)."""
    import pandas as pd
    start_time = time.time()
    tracemalloc.start()

    df = pd.read_csv(filepath)
    df = df[(df["balance"] >= 0) & (df["amount"] >= 0)]
    df["balance"] = df.apply(
        lambda row: row["balance"] + row["amount"] if row["operation"] == "deposit"
        else (row["balance"] - row["amount"] if row["balance"] - row["amount"] >= 0 else row["balance"]),
        axis=1
    )
    total_sum = df["balance"].sum()

    current, peak = tracemalloc.get_traced_memory()
    tracemalloc.stop()
    elapsed = time.time() - start_time
    return elapsed, peak, total_sum


def measure_lazy_processing(filepath: str):
    """Lazy evaluation: потокова обробка без матеріалізації колекцій."""
    start_time = time.time()
    tracemalloc.start()

    pipeline = run_lazy_pipeline(filepath, min_balance=100.0)
    total_sum = sum(rec["balance"] for rec in pipeline)
```

```python
    current, peak = tracemalloc.get_traced_memory()
    tracemalloc.stop()
    elapsed = time.time() - start_time
    return elapsed, peak, total_sum


if __name__ == "__main__":
    print("=== БАНКІВСЬКА СИСТЕМА: ПОТОКОВА ОБРОБКА ДАНИХ (ЛР №3) ===")

    # 1. Генерація датасету (100 000+ записів)
    print("[1] Генерація тестового набору даних (100 000 записів)...")
    generate_large_dataset(DATA_PATH, total_records=100_000)
    print(f"   Файл створено: {DATA_PATH} (розмір: {os.path.getsize(DATA_PATH) / 1024 / 1024:.2f} MB)\n")

    # 2. Вибірка перших записів через islice
    print("[2] Демонстрація роботи lazy pipeline та вибірки перших 5 записів (islice):")
    pipeline = run_lazy_pipeline(DATA_PATH)
    first_records = list(islice(pipeline, 5))
    for idx, rec in enumerate(first_records, 1):
        print(f"   {idx}. Рахунок: {rec['account_number']} | Клієнт: {rec['client']} | Баланс: {rec['balance']} | Операція: {rec['operation']} {rec['amount']}")
    print()

    # 3. Використання itertools (groupby, accumulate, pairwise, chain)
    print("[3] Демонстрація інструментів itertools:")
    pipeline_it = run_lazy_pipeline(DATA_PATH)
    rec_list = sorted(list(pipeline_it), key=lambda x: x["client"])

    # groupby
    client_groups = {client: len(list(group)) for client, group in groupby(rec_list, key=lambda x: x["client"])}
    print(f"   Кількість записів по клієнтах (groupby): {client_groups}")

    # accumulate та pairwise
    balances = [r["balance"] for r in rec_list[:10]]
    acc_balances = list(accumulate(balances))
    diffs = [round(curr - prev, 2) for prev, curr in pairwise(balances)]
    print(f"   Накопичувальні баланси (accumulate): {acc_balances[:5]}")
    print(f"   Різниці сусідніх транзакцій (pairwise): {diffs[:5]}")
```

```
print(f"   Різниці сусідніх транзакцій (pairwise): {diffs[:5]}")
print()

# 4. Порівняння Eager та Lazy підходів
print("[4] Порівняння продуктивності (Eager vs Lazy):")
eager_time, eager_peak, eager_sum = measure_eager_processing(DATA_PATH)
lazy_time, lazy_peak, lazy_sum = measure_lazy_processing(DATA_PATH)

print(f"   [Eager] Час: {eager_time:.4f} сек | Пікова пам'ять: {eager_peak / 1024 / 1024:.2f} MB | Сума: {eager_sum}")
print(f"   [Lazy]  Час: {lazy_time:.4f} сек | Пікова пам'ять: {lazy_peak / 1024 / 1024:.2f} MB | Сума: {lazy_sum}")
print("\nВиконання програми завершено успішно!")
```

3. Робота з Git

Для відстеження змін у другій лабораторній роботі було виконано серію логічних комітів:

```
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git init
Reinitialized existing Git repository in C:/Users/Maximus/Desktop/Професійний пітон/bank_system/.git/
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git add src/bank_system/iterators.py
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git commit -m "feat(iterators): реалізовано власний ітератор AccountRangeIterator та рекурсивний генератор з yield fro
[main 8f6ded5] feat(iterators): реалізовано власний ітератор AccountRangeIterator та рекурсивний генератор з yield from
 1 file changed, 26 insertions(+)
 create mode 100644 src/bank_system/iterators.py
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git add src/bank_system/streaming.py
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git commit -m "feat(streaming): додано генератор тестового датасету, потоковий reader та lazy pipeline"
[main f01d91b] feat(streaming): додано генератор тестового датасету, потоковий reader та lazy pipeline
 1 file changed, 97 insertions(+)
 create mode 100644 src/bank_system/streaming.py
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git add src/bank_system/main.py
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git commit -m "feat(main): інтегровано аналітику itertools, пакетну обробку та порівняння eager/lazy підходів"
[main 94e177e] feat(main): інтегровано аналітику itertools, пакетну обробку та порівняння eager/lazy підходів
 1 file changed, 73 insertions(+), 117 deletions(-)
```

Перевірка історії комітів (git log --oneline):

```
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git log --oneline
94e177e (HEAD -> main, origin/main) feat(main): інтегровано аналітику itertools, пакетну обробку та порівняння eager/lazy підходів
f01d91b feat(streaming): додано генератор тестового датасету, потоковий reader та lazy pipeline
8f6ded5 feat(iterators): реалізовано власний ітератор AccountRangeIterator та рекурсивний генератор з yield from
c7b759c Збереження поточного стану проєкту
5c2f55e Збереження поточного стану проєкту
f792e90 Звіт для 2 лабораторної
651213d Add README with project details and structure
0e5a2aa Оновлено main.py з інтерактивним меню для ЛР2
a155460 Додано обробку індексів, замикання, сортування та аналітику з defaultdict і *args
64eab4b Додано модуль data.py з **kwargs та модуль декораторів decorators.py
d7bef70 Додано звіт до лабораторної роботи №1
53d3b57 Додано точку входу та документацію застосунку Bank System
603e965 Додано модель банківського рахунку та бізнес-логіку
454af57 Створення структури проєкту
```

Індивідуальне завдання (Варіант 4)

Індивідуальне завдання (Варіант 4). Потокова обробка банківських рахунків

Обробляються дані банківської системи: номери рахунків, клієнти, баланси та операції у форматі account_number,client,balance,operation,amount.

Розроблено конвеєр потокової обробки, який виконує такі завдання:

1. Зчитування даних: генератор stream_csv_reader() читає CSV-файл рядок за рядком.
2. Валідація: parse_and_validate_records() перевіряє дані та відсіює некоректні записи.
3. Фільтрація: відбір рахунків за клієнтом або мінімальним балансом.
4. Обробка операцій: поповнення та списання коштів із перевіркою достатності балансу.
5. Вибірка записів: використання itertools.islice для отримання перших *N* записів.
6. Агрегація: обчислення загального балансу за один прохід.
7. Пакетна обробка: поділ потоку даних на пакети фіксованого розміру.

Результат: реалізовано потокову обробку банківських даних із використанням генераторів, фільтрації та агрегації для зменшення споживання пам’яті.

Експериментальна частина

Під час запуску головного скрипта виконується генерація датасету на 100 000 записів, демонстрація роботи ледачого конвеєра та порівняння продуктивності:

```
=== БАНКІВСЬКА СИСТЕМА: ПОТОКОВА ОБРОБКА ДАНИХ (ЛР №3) ===
[1] Генерація тестового набору даних (100 000 записів)...
   Файл створено: data\input.csv (розмір: 3.92 MB)

[2] Демонстрація роботи lazy pipeline та вибірки перших 5 записів (islice):
   1. Рахунок: ACC-1000 | Клієнт: Client_A | Баланс: 1010.0 | Операція: deposit 10.0
   2. Рахунок: ACC-1001 | Клієнт: Client_B | Баланс: 994.0 | Операція: withdraw 13.0
   3. Рахунок: ACC-1002 | Клієнт: Client_C | Баланс: 1030.0 | Операція: deposit 16.0
   4. Рахунок: ACC-1003 | Клієнт: Client_D | Баланс: 1002.0 | Операція: withdraw 19.0
   5. Рахунок: ACC-1004 | Клієнт: Client_E | Баланс: 1050.0 | Операція: deposit 22.0

[3] Демонстрація інструментів itertools:
   Кількість записів по клієнтах (groupby): {'Client_A': 20000, 'Client_B': 20000, 'Client_C': 20000, 'Client_D': 20000, 'Client_E': 20000}
   Накопичувальні баланси (accumulate): [1010.0, 2020.0, 3130.0, 4180.0, 5390.0]
   Різниці сусідніх транзакцій (pairwise): [0.0, 100.0, -60.0, 160.0, -120.0]

[4] Порівняння продуктивності (Eager vs Lazy):
   [Eager] Час: 1.0523 сек | Пікова пам'ять: 28.90 MB | Сума: 2599900000.0
   [Lazy]  Час: 0.5843 сек | Пікова пам'ять: 0.18 MB | Сума: 2599900000.0
```

Таблиця виконання вимог

Ось детальна **таблиця виконання вимог** для твого звіту до лабораторної роботи №3, адаптована спеціально під індивідуальне завдання (Варіант 4 — потокова обробка банківських рахунків):

Вимога лабораторної роботи / Індивідуального завдання	**Стан виконання**	**Деталі реалізації у проєкті**
Власний ітератор (Iterator class)	Виконано	Реалізовано клас AccountRangeIterator у модулі iterators.py з підтримкою протоколу ітерації (__iter__ / __next__).
Генераторні функції та yield	Виконано	Задіяно для потокового читання файлу рядок за рядком (stream_csv_reader) та на етапах конвеєра в streaming.py.
Рекурсивний генератор із yield from	Виконано	Реалізовано функцію recursive_flatten_operations у iterators.py для розгортання вкладених структур даних.
Генераторні вирази (Generator Expressions)	Виконано	Використовуються для ледачих обчислень (*lazy evaluation*) в аналітичних підрахунках та сумуванні балансів.

Вимога лабораторної роботи / Індивідуального завдання	**Стан виконання**	**Деталі реалізації у проєкті**
Використання модуля itertools	Виконано	Задіяно функціонал islice (зрізи), groupby (групування за клієнтами), accumulate (накопичувальні суми), pairwise (аналіз різниць транзакцій) та chain (об'єднання потоків).
Потоковий конвеєр (Lazy Pipeline)	Виконано	Зібрано повний ланцюжок: source → parsing → validation → filtering → transformation → aggregation.
Фільтрація та валідація рахунків	Виконано	Фільтрація за клієнтом та порогом балансу, а також валідація коректності полів на льоту.
Контроль від'ємного балансу	Виконано	У процесі обробки операцій списання (withdraw) перевіряється умова недопущення від'ємного балансу рахунку.
Batch Processing (Пакетна обробка)	Виконано	Реалізовано групування потоку у пакети заданого розміру за допомогою islice для порційної обробки.
Обробка набору від 100 000 записів	Виконано	Автоматична генерація та потокова обробка тестового датасету обсягом 100 000+ рядків.
Порівняння Eager та Lazy підходів	Виконано	Проведено вимірювання часу виконання та пікового споживання пам'яті (*tracemalloc*) між Pandas (Eager) та генераторами (Lazy).

Контрольні запитання

1. Що таке iterable?

Об’єкт, елементи якого можна перебирати, наприклад список або рядок.

2. Що таке iterator?
Об’єкт, який повертає наступний елемент послідовності.
3. Чим iterable відрізняється від iterator?
Iterable можна перетворити на ітератор, а iterator безпосередньо видає елементи.
4. Для чого використовується iter()?
Для отримання ітератора з iterable-об’єкта.
5. Для чого використовується next()?
Для отримання наступного елемента ітератора.
6. Що робить __iter__()?
Повертає ітератор об’єкта.
7. Що робить __next__()?
Повертає наступний елемент або сигналізує про завершення перебору.
8. Що таке StopIteration?
Виняток, який повідомляє, що елементи ітератора закінчилися.
9. Як цикл for використовує iterator protocol?
Отримує ітератор, послідовно викликає next() і зупиняється при StopIteration.
10. Чому iterator називають one-shot object?
Тому що після вичерпання його елементи не можна перебрати повторно без створення нового ітератора.
11. Чому iter(iterator) is iterator?
Тому що ітератор повертає сам себе при виклику iter().
12. Чи можна повторно використати exhausted generator?
Ні, потрібно створити новий генератор.
13. Що таке generator function?
Функція, яка використовує yield і повертає генератор.
14. Яку роль виконує yield?
Повертає значення та призупиняє виконання функції.
15. Чим yield відрізняється від return?
yield призупиняє функцію, а return завершує її.
16. Коли фактично починає виконуватися generator function?
Коли запитують перший елемент, наприклад через next().
17. Що зберігає generator між викликами next()?
Поточний стан виконання, локальні змінні та позицію в коді.
18. Що таке generator expression?
Короткий спосіб створити генератор через вираз у дужках.
19. Чим generator expression відрізняється від list comprehension?
Генератор обчислює елементи поступово, а список створює всі одразу.
20. Що таке lazy evaluation?
Обчислення значень лише тоді, коли вони потрібні.
21. Що таке eager evaluation?
Обчислення всіх значень одразу.
22. Які переваги lazy evaluation?
Менше споживання пам’яті та можливість швидко отримати перший результат.

23. Які недоліки lazy evaluation?
Складніше налагодження, а повторний перебір часто потребує нового генератора.
24. Що таке streaming?
Обробка даних поступово, без завантаження всього набору в пам’ять.
25. Чому streaming важливий для великих файлів?
Він дозволяє обробляти великі обсяги даних із меншими витратами пам’яті.
26. Чому read() може бути проблемою для multi-GB file?
Він може завантажити весь файл у пам’ять і спричинити її нестачу.
27. Чому file object є iterable?
Тому що його можна перебирати рядок за рядком.
28. Для чого використовується yield from?
Для передачі елементів іншого iterable або генератора.
29. Що таке itertools?
Модуль Python із засобами для ефективної роботи з ітераторами.
30. Для чого itertools.count()?
Для створення нескінченної послідовності чисел.
31. Чому count() потрібно обмежувати?
Тому що він нескінченний і без обмеження не завершиться.
32. Для чого islice()?
Для отримання певної кількості елементів ітератора.
33. Для чого chain()?
Для послідовного об’єднання кількох iterable-об’єктів.
34. Як працює takewhile()?
Повертає елементи, поки умова істинна.
35. Як працює dropwhile()?
Пропускає елементи, поки умова істинна, а потім повертає решту.
36. Для чого accumulate()?
Для обчислення накопичувального результату, наприклад поточної суми.
37. Для чого pairwise()?
Для отримання сусідніх пар елементів.
38. Як працює groupby()?
Групує послідовні елементи за спільним ключем.
39. Чому перед groupby() часто потрібне сортування?
Щоб однакові ключі йшли поруч і утворювали правильні групи.
40. Що робить product()?
Створює декартів добуток наборів значень.
41. Чим combinations() відрізняється від permutations()?
combinations() не враховує порядок, а permutations() враховує.
42. Чому combinatorial iterators можуть бути небезпечними навіть при lazy execution?
Кількість можливих комбінацій може бути надзвичайно великою.
43. Що таке batching?

Поділ даних на невеликі пакети для обробки.
44. Які переваги batching?
Контроль використання пам’яті та зручніша обробка великих наборів даних.
45. Що таке lazy pipeline?
Послідовність етапів обробки, які виконуються поступово за потреби.
46. Які етапи може містити pipeline?
Джерело даних, перетворення, фільтрацію та кінцеве отримання результату.
47. Що є source у streaming pipeline?
Джерело даних, наприклад файл або генератор.
48. Що є transformation?
Етап, який змінює або фільтрує дані.
49. Що є sink?
Кінцевий отримувач результату, наприклад файл або база даних.
50. Чи гарантує generator менший total execution time?
Ні, генератор економить пам’ять, але не обов’язково час виконання.
51. Яка основна memory advantage generator?
Він не зберігає всі результати одночасно в пам’яті.
52. Що таке time to first result? Час від початку роботи до отримання першого результату.
53. Чому lazy pipeline добре підтримує early termination?
Тому що можна зупинити обробку, щойно потрібні результати отримано.
54. Чому exception у generator може виникати із затримкою?
Тому що код генератора виконується лише під час запиту елементів.
55. Чому list(generator) може бути проблемою?
Тому що всі елементи завантажуються в пам’ять одночасно.
56. Що трапиться при list(count())?
Список ніколи не завершить створення, оскільки count() нескінченний.
57. Як generator може впливати на lifecycle file resource?
Файл може залишатися відкритим, поки генератор його використовує.
58. Як тестувати невеликий generator?
Перетворити його результат на список і порівняти з очікуваним.
59. Як тестувати infinite generator?
Обмежити кількість елементів за допомогою islice().
60. Коли краще використовувати list, а коли generator?
Список — коли потрібен повторний доступ до всіх елементів, генератор — для потокової обробки та економії пам’яті.

Висновок:
У ході виконання лабораторної роботи №3 було успішно опановано ітераційну модель мови Python, розширені механізми роботи з ітераторами, генераторами (yield, yield from) та виразами-генераторами на прикладі індивідуального варіанта системи банківських рахунків.

На відміну від традиційного підходу з повним завантаженням колекцій у пам'ять (*eager evaluation*), було побудовано повноцінний потоковий конвеєр (*lazy pipeline*), який здійснює обробку великих наборов даних (понад 100 000 записів) рядок за рядком та пакетами (*batch processing*) без матеріалізації проміжних структур. Це дозволило суттєво знизити пікове споживання оперативної пам'яті (*RAM*) та забезпечити миттєвий час отримання перших результатів (*time-to-first-result*).

Також було практично застосовано вбудовані будівельні блоки модуля itertools (islice, groupby, accumulate, pairwise, chain), реалізовано власний ітератор (AccountRangeIterator) із дотриманням протоколу ітерації та рекурсивний генератор для розгортання вкладених структур даних.

Проведені експериментальні вимірювання за допомогою бібліотек time та tracemalloc підтвердили переваги ледачих обчислень (*lazy evaluation*) над підходами з повним завантаженням (*eager evaluation*) при роботі з великими обсягами інформації.